STANDARD OPERATING PROCEDURE

Production Database Migration

AWS Database Migration Service (DMS)

Oracle Source → Amazon RDS for Oracle

Document Info	Details
Document Type	Standard Operating Procedure (SOP)
Classification	CONFIDENTIAL — Internal Use Only
Version	2.0
Effective Date	May 2026
Review Cycle	Quarterly or after each major migration
Owner	Database Engineering / Cloud Infrastructure
Approvers	DBA Lead, Change Advisory Board (CAB)

WARNING: This SOP governs production database operations. All steps must be followed in sequence. Any deviation requires written approval from the CAB. A rollback plan must be validated before cutover commences.

Table of Contents

1. Document Control & Approvals 3
2. Overview & Scope 4
3. Roles & Responsibilities (RACI) 5
4. Pre-Migration Prerequisites 6
5. Phase 1 — Environment Preparation 7
6. Phase 2 — DMS Task Configuration 9
7. Phase 3 — Pre-Cutover Validation & Checklists 11
8. Phase 4 — Cutover Execution 15
9. Phase 5 — Post-Cutover Validation 17
10. Phase 6 — Hypercare & Monitoring 20
11. Rollback Plan 21
12. SQL Reference — All Validation Queries 22
13. Troubleshooting Guide 27
14. Communication & Escalation Plan 29
15. Sign-off & Approval 30

1. Document Control & Approvals

1.1 Revision History

Version	Date	Author	Change Summary
1.0	Jan 2026	DBA Team	Initial draft
1.1	Mar 2026	Cloud Infra	Added CDC validation steps
2.0	May 2026	DBA Lead	Full revision — added Phase 4 cutover playbook, rollback SOP

1.2 Document Approvals

Role	Name	Signature	Date
DBA Lead			
Cloud Infrastructure Lead			
Application Owner			
Change Advisory Board			
CISO / Security Review			

2. Overview & Scope

2.1 Purpose

This SOP provides the definitive operational procedure for migrating a production Oracle database to Amazon RDS for Oracle using AWS Database Migration Service (DMS). It covers all phases from environment preparation through post-cutover hypercare, including every SQL validation query, checklist, and rollback procedure required for a safe, auditable production migration.

2.2 Scope

- Source: On-premises Oracle Database 11g / 12c / 19c (or EC2-hosted)
- Target: Amazon RDS for Oracle (Single-AZ pre-cutover; Multi-AZ post-cutover)
- Migration method: AWS DMS Full Load + Change Data Capture (CDC)
- Includes: Schema objects, data, sequences, constraints, indexes, stored procedures
- Excludes: Oracle-specific features not supported by RDS (e.g., external tables, directory objects)

2.3 Migration Architecture

Layer	Component	Notes
Source	Oracle DB (on-prem / EC2)	Must have Supplemental Logging enabled
Network	AWS Direct Connect or Site-to-Site VPN	Low latency < 10ms recommended for CDC
Replication Engine	AWS DMS Replication Instance	r5.2xlarge or larger for prod migration
Target	Amazon RDS for Oracle	Same or higher Oracle version as source
Monitoring	Amazon CloudWatch + DMS Console	Alerts configured for all critical metrics
Storage	Amazon S3	DMS logs, validation reports, task exports

2.4 Migration Timeline Overview

Phase	Activity	Duration (Est.)	Responsible
Pre-Migration	Prerequisites, environment prep	1-2 weeks	DBA + Cloud
Phase 1	Source/target configuration	2-3 days	DBA
Phase 2	DMS task setup, full load	Hours–days	DBA + Cloud
Phase 3	Validation & cutover readiness	1-2 days	DBA + App
Phase 4	Production cutover execution	2-4 hours	All teams
Phase 5	Post-cutover validation	4-8 hours	DBA + App
Phase 6	Hypercare & handover	1-2 weeks	DBA + Ops

3. Roles & Responsibilities (RACI)

Activity	DBA Lead	Cloud Eng	App Owner	Sec/Audit	CAB
Migration planning & SOP approval	A	C	C	C	A
Source DB configuration	R	I	I	I	I
AWS environment setup	C	R	I	I	I
DMS task creation & configuration	R	R	I	I	I
Full load monitoring	R	R	I	I	I
Data validation (pre-cutover)	R	C	C	I	I
Application testing on RDS	C	I	R	I	I
Change request approval (CAB)	C	C	C	I	A
Cutover execution	R	R	C	I	I
Rollback decision & execution	A	C	C	I	I
Post-cutover sign-off	A	R	R	C	A

R = Responsible | A = Accountable | C = Consulted | I = Informed

4. Pre-Migration Prerequisites

4.1 Change Management

- ☐ Change Request (CR) raised and approved by Change Advisory Board (CAB)
- ☐ Maintenance window confirmed with all stakeholders (app teams, business, ops)
- ☐ Rollback window defined (recommend minimum 2-hour rollback window post-cutover)
- ☐ Communication sent to all affected teams at least 5 business days in advance
- ☐ Incident response team on standby during cutover window

4.2 Source Oracle Database

- ☐ Oracle version documented (SELECT * FROM v$version)
- ☐ Archive log mode enabled (SELECT log_mode FROM v$database)
- ☐ Supplemental logging enabled at DB and table level
- ☐ DMS user created with all required privileges
- ☐ Source DB schema export (expdp) taken as backup
- ☐ AWR / Statspack baseline report captured
- ☐ All active sessions and long-running transactions documented

4.3 AWS Environment

- ☐ VPC, subnets, and security groups configured for DMS replication instance
- ☐ DMS replication instance created (r5.2xlarge minimum for prod)
- ☐ Source Oracle endpoint created and connection tested successfully
- ☐ Target RDS Oracle endpoint created and connection tested successfully
- ☐ RDS parameter group configured to match source Oracle init.ora settings
- ☐ RDS option group configured (Timezone, character set matching source)
- ☐ RDS Multi-AZ will be enabled AFTER cutover (not during migration)
- ☐ S3 bucket created for DMS logs and validation reports
- ☐ CloudWatch alarms configured for DMS metrics
- ☐ AWS Secrets Manager updated with RDS credentials

4.4 Application Readiness

- ☐ Application team has tested connectivity to RDS in staging/UAT
- ☐ All stored procedures, functions, and packages compiled on RDS target
- ☐ Application configuration management updated with RDS endpoint (pending cutover)
- ☐ Database connection pool settings reviewed for RDS (max connections = PROCESSES parameter)

5. Phase 1 — Environment Preparation

5.1 DMS User Setup on Source Oracle

Create a dedicated DMS user on the source Oracle database with minimum required privileges:

```
-- Create DMS user
CREATE USER dms_user IDENTIFIED BY '<strong_password>';
GRANT CREATE SESSION TO dms_user;
GRANT SELECT ANY TABLE TO dms_user;
GRANT SELECT ANY TRANSACTION TO dms_user;
GRANT SELECT ON V_$ARCHIVED_LOG TO dms_user;
GRANT SELECT ON V_$LOG TO dms_user;
GRANT SELECT ON V_$LOGFILE TO dms_user;
GRANT SELECT ON V_$DATABASE TO dms_user;
GRANT SELECT ON V_$THREAD TO dms_user;
GRANT SELECT ON V_$PARAMETER TO dms_user;
GRANT SELECT ON V_$NLS_PARAMETERS TO dms_user;
GRANT SELECT ON V_$TIMEZONE_NAMES TO dms_user;
GRANT SELECT ON ALL_INDEXES TO dms_user;
GRANT SELECT ON ALL_OBJECTS TO dms_user;
GRANT SELECT ON ALL_TABLES TO dms_user;
GRANT SELECT ON ALL_USERS TO dms_user;
GRANT SELECT ON ALL_CATALOG TO dms_user;
GRANT SELECT ON ALL_CONSTRAINTS TO dms_user;
GRANT SELECT ON ALL_CONS_COLUMNS TO dms_user;
GRANT SELECT ON ALL_TAB_COLS TO dms_user;
GRANT SELECT ON ALL_IND_COLUMNS TO dms_user;
GRANT SELECT ON ALL_ENCRYPTED_COLUMNS TO dms_user;
GRANT SELECT ON ALL_LOG_GROUPS TO dms_user;
GRANT SELECT ON ALL_TAB_PARTITIONS TO dms_user;
GRANT SELECT ON SYS.DBA_REGISTRY TO dms_user;
GRANT SELECT ON SYS.OBJ$ TO dms_user;
GRANT SELECT ON DBA_TABLESPACES TO dms_user;
GRANT SELECT ON DBA_OBJECTS TO dms_user;
GRANT SELECT ON SYS.ENC$ TO dms_user;
-- Required for LogMiner (CDC)
GRANT EXECUTE ON DBMS_LOGMNR TO dms_user;
GRANT EXECUTE ON DBMS_LOGMNR_D TO dms_user;
GRANT SELECT ON V_$LOGMNR_LOGS TO dms_user;
GRANT SELECT ON V_$LOGMNR_CONTENTS TO dms_user;
GRANT SELECT ON V_$LOGMNR_PARAMETERS TO dms_user;
```

5.2 Enable Supplemental Logging

```
-- Check current supplemental logging status
SELECT supplemental_log_data_min  AS min_logging,
       supplemental_log_data_pk   AS pk_logging,
       supplemental_log_data_ui   AS ui_logging,
       supplemental_log_data_fk   AS fk_logging,
       supplemental_log_data_all  AS all_logging
FROM   v$database;
```

```
-- Enable minimum supplemental logging (REQUIRED for CDC)
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;

-- Enable primary key supplemental logging (RECOMMENDED)
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (PRIMARY KEY) COLUMNS;

-- Enable for specific schema (if not enabling at DB level)
-- Run for each table that will be replicated via CDC:
ALTER TABLE schema_name.table_name ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;

-- Verify it is active
SELECT log_mode, supplemental_log_data_min FROM v$database;
-- Expected: ARCHIVELOG, YES
```

5.3 RDS Target Parameter Group Settings

Parameter	Recommended Value	Reason
enable_goldengate_replication	TRUE	Required for LogMiner / CDC target
undo_retention	3600	Prevent ORA-01555 during full load
log_buffer	10485760	Increase redo log buffer (10MB)
db_files	500	Allow enough data files
open_cursors	1000	Prevent cursor exhaustion
processes	500	Max concurrent connections
sessions	1000	Sessions = 1.5 * processes + 22
nls_characterset	Match source	Avoid character set conversion errors
time_zone	Match source	Prevent TIMESTAMP WITH TZ issues

6. Phase 2 — DMS Task Configuration

6.1 DMS Task Settings (Recommended for Production)

Setting Category	Setting	Recommended Value
Migration Type	Migration type	Migrate existing data and replicate ongoing changes
Target Table Prep	Target table preparation mode	Do nothing (schema pre-created via SCT or DDL)
LOB Settings	LOB column settings	Limited LOB mode (32KB) OR Full LOB mode for large CLOBs
Validation	Enable validation	YES — mandatory for prod
Logging	CloudWatch logging	YES — enable all log levels
Error Handling	Escalate error as warning	NO — treat errors as errors
Full Load	Max full load sub-tasks	8 (tune based on replication instance size)
Full Load	Transaction consistency timeout	600 seconds
CDC	CDC start mode	Use source transaction log (LogMiner)
CDC	CDC stop position	Leave empty (continuous replication)
Parallel Load	Parallel load for partitioned tables	YES — for large partitioned tables

6.2 Table Mapping Rules

```
{
  "rules": [
    {
      "rule-type": "selection",
      "rule-id": "1",
      "rule-name": "include-all-tables",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA",
        "table-name": "%"
      },
      "rule-action": "include"
    },
    {
      "rule-type": "transformation",
      "rule-id": "2",
      "rule-name": "convert-schema-to-upper",
      "rule-action": "convert-uppercase",
      "rule-target": "schema",
      "object-locator": { "schema-name": "%" }
    },
```

```
    {
      "rule-type": "selection",
      "rule-id": "99",
      "rule-name": "exclude-temp-tables",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA",
        "table-name": "TEMP_%"
      },
      "rule-action": "exclude"
    }
  ]
}
```

6.3 Full Load Monitoring

Monitor the following metrics in CloudWatch during full load:

CloudWatch Metric	Healthy Range	Alert If
FullLoadThroughputRowsSource	> 5,000 rows/sec	Drops to 0 for > 5 minutes
FullLoadThroughputRowsTarget	> 5,000 rows/sec	Drops to 0 for > 5 minutes
CPUUtilization (Rep Instance)	< 80%	> 85% sustained for 10 min
FreeableMemory (Rep Instance)	> 1 GB	< 500 MB
FreeStorageSpace (Rep Instance)	> 20 GB	< 10 GB
NetworkReceiveThroughput	Within link capacity	Approaching saturation

7. Phase 3 — Pre-Cutover Validation & Checklists

CRITICAL: All validations in this section must PASS before the cutover window is opened. Any FAIL item must be resolved or formally accepted by the DBA Lead and Application Owner.

7.1 Row Count Validation Checklist

```
-- QUERY 1: Source — Estimated row counts (fast)
SELECT table_name,
       num_rows            AS estimated_rows,
       last_analyzed       AS stats_date
FROM   user_tables
ORDER BY table_name;

-- QUERY 2: Source — Exact row counts (run per table during low traffic)
-- Generate dynamic SQL for all tables:
SELECT 'SELECT ''' || table_name || ''' tbl, COUNT(*) cnt FROM ' || table_name || '
UNION ALL'
FROM   user_tables ORDER BY table_name;

-- QUERY 3: Comparison helper — run on both source and target
SELECT t.table_name,
       t.num_rows          AS dict_count,
       (SELECT COUNT(*) FROM user_tables WHERE table_name = t.table_name) AS live_check
FROM   user_tables t
ORDER BY t.table_name;

-- QUERY 4: SCN-consistent count on source
SELECT current_scn FROM v$database;
-- Then query at that SCN:
SELECT COUNT(*) FROM schema.critical_table AS OF SCN &scn_value;
```

Table Name	Source Count	Target Count	Diff	Result
ALL TABLES	[from QUERY 1/2]	[from target]	0	
Critical Table 1				
Critical Table 2				
Critical Table 3				

7.2 DMS Built-in Validation Checklist

```
-- QUERY 5: Check DMS validation via AWS CLI
aws dms describe-table-statistics \
  --replication-task-arn arn:aws:dms:region:acct:task:TASK_ID \
  --query
'TableStatistics[*].{Table:TableName,FullLoad:FullLoadRows,Errors:FullLoadErrorRows,ValidationState:Vali
```

```
-- QUERY 6: Tables with validation failures
aws dms describe-table-statistics \
  --replication-task-arn arn:aws:dms:region:acct:task:TASK_ID \
  --filters Name=validation-state,Values=mismatched-records,error \
  --query 'TableStatistics[*].{Table:TableName,Mismatch:ValidationSuspendedRecords}'
```

Validation Check	Expected	Actual	Result
DMS Validation: Validated tables	All tables		
DMS Validation: Mismatched records	0		
DMS Validation: Failed rows	0		
DMS Validation: Pending records	0		
DMS Task errors	0		

7.3 Schema Object Validation

```
-- QUERY 7: Object count by type (run on source AND target, compare)
SELECT object_type,
       COUNT(*)                                                AS total_count,
       SUM(CASE WHEN status = 'VALID'   THEN 1 ELSE 0 END) AS valid_count,
       SUM(CASE WHEN status = 'INVALID' THEN 1 ELSE 0 END) AS invalid_count
FROM   user_objects
GROUP BY object_type
ORDER BY object_type;

-- QUERY 8: List all INVALID objects on target (must be 0 before cutover)
SELECT object_name, object_type, status, last_ddl_time
FROM   user_objects
WHERE  status = 'INVALID'
ORDER BY object_type, object_name;

-- QUERY 9: Recompile all invalid objects
EXEC DBMS_UTILITY.COMPILE_SCHEMA(schema => USER, compile_all => FALSE);

-- QUERY 10: Constraint validation
SELECT constraint_type, status, validated, COUNT(*) AS cnt
FROM   user_constraints
GROUP BY constraint_type, status, validated
ORDER BY constraint_type;

-- QUERY 11: Disabled constraints that must be re-enabled
SELECT table_name, constraint_name, constraint_type, status, validated
FROM   user_constraints
WHERE  status = 'DISABLED'
ORDER BY table_name;
```

```
-- QUERY 12: Trigger status check
SELECT trigger_name, table_name, triggering_event, status
FROM   user_triggers
ORDER BY table_name;
```

Schema Check	Source Count	Target Count	Result
Total Tables			
Total Views			
Total Procedures			
Total Functions			
Total Packages			
Total Triggers			
INVALID Objects	0 target	Must be 0	
DISABLED Constraints	0 target	Must be 0	
Indexes UNUSABLE	0 target	Must be 0	

7.4 Data Integrity Validation

```
-- QUERY 13: Checksum comparison per table (run on source AND target)
SELECT 'ORDERS' AS tbl,
       COUNT(*)                                          AS row_count,
       SUM(ORA_HASH(TO_CHAR(order_id) || status ||
           TO_CHAR(order_date,'YYYYMMDDHH24MISS')))      AS chksum
FROM   schema.orders
UNION ALL
SELECT 'CUSTOMERS',
       COUNT(*),
       SUM(ORA_HASH(TO_CHAR(customer_id) || customer_name || email))
FROM   schema.customers;

-- QUERY 14: Boundary row check (first and last by PK)
SELECT * FROM schema.orders ORDER BY order_id ASC  FETCH FIRST 10 ROWS ONLY;
SELECT * FROM schema.orders ORDER BY order_id DESC FETCH FIRST 10 ROWS ONLY;

-- QUERY 15: NULL count comparison (critical columns only)
SELECT 'orders.status NULL count' AS check_name,
       COUNT(*) AS null_count
FROM   schema.orders WHERE status IS NULL
UNION ALL
SELECT 'customers.email NULL count',
       COUNT(*) FROM schema.customers WHERE email IS NULL;

-- QUERY 16: Referential integrity check (run on TARGET)
```

```
-- Orphaned order_items (no parent order)
SELECT COUNT(*) AS orphan_order_items
FROM   schema.order_items oi
WHERE  NOT EXISTS (SELECT 1 FROM schema.orders o WHERE o.order_id = oi.order_id);
-- Expected: 0
```

7.5 Sequence & Index Validation

```
-- QUERY 17: Sequence current values (run on source)
SELECT sequence_name,
       last_number     AS current_value,
       increment_by,
       max_value,
       cycle_flag,
       cache_size
FROM   user_sequences
ORDER BY sequence_name;

-- QUERY 18: Max PK values per table (compare to sequence last_number)
-- Run for each sequence-backed table:
SELECT MAX(order_id)    AS max_order_id    FROM schema.orders;
SELECT MAX(customer_id) AS max_customer_id FROM schema.customers;

-- QUERY 19: Advance sequence on target to be > source max PK
-- Step 1: Determine gap
-- Step 2: Increment sequence by gap+10000 for safety
ALTER SEQUENCE schema.orders_seq INCREMENT BY 100000;
SELECT schema.orders_seq.NEXTVAL FROM DUAL;   -- consume one value
ALTER SEQUENCE schema.orders_seq INCREMENT BY 1;
-- Verify:
SELECT last_number FROM user_sequences WHERE sequence_name = 'ORDERS_SEQ';

-- QUERY 20: Index status check
SELECT index_name, table_name, index_type, status, uniqueness
FROM   user_indexes
WHERE  status != 'VALID'
ORDER BY table_name;
-- Expected: 0 rows (all indexes VALID)

-- QUERY 21: Rebuild unusable indexes
ALTER INDEX schema.index_name REBUILD ONLINE;
```

7.6 CDC Lag Validation

```
-- QUERY 22: Check CDC lag via AWS CLI
aws cloudwatch get-metric-statistics \
  --namespace AWS/DMS \
  --metric-name CDCLatencyTarget \
  --dimensions Name=ReplicationInstanceIdentifier,Value=YOUR-REP-INSTANCE \
  --start-time $(date -u -d '30 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
  --period 60 --statistics Average

-- QUERY 23: Verify CDC is current — test transaction
-- On SOURCE:
INSERT INTO schema.dms_cdc_test (test_id, test_val, inserted_at)
VALUES (SYS_GUID(), 'CDC_PRECHECK_' || TO_CHAR(SYSDATE,'YYYYMMDDHH24MISS'),
SYSTIMESTAMP);
COMMIT;

-- On TARGET (within 60 seconds):
SELECT * FROM schema.dms_cdc_test
WHERE  test_val LIKE 'CDC_PRECHECK_%'
ORDER BY inserted_at DESC FETCH FIRST 5 ROWS ONLY;
-- Row must appear within configured CDC latency threshold
```

Pre-Cutover Gate Check	Criteria	Result
Row counts match across all tables	Exact match (0 diff)	
DMS validation 0 mismatched records	= 0	
DMS validation 0 failed rows	= 0	
0 INVALID objects on target RDS	= 0	
All constraints ENABLED and VALIDATED	= 0 disabled	
All indexes VALID	= 0 unusable	
Sequences advanced above source max PK	Confirmed	
CDC latency < 60 seconds	< 60 sec	
CDC test transaction replicated	Verified	
Application smoke test on RDS PASSED	Pass	
RDS backup enabled and verified	Confirmed	
CAB change window open	Approved	
Rollback plan reviewed and ready	Confirmed	
DBA Lead sign-off on GO decision	Signed	

8. Phase 4 — Cutover Execution

PRODUCTION CUTOVER — This section is executed LIVE during the maintenance window. Follow steps in exact sequence. Time each step. If any step fails, invoke the Rollback Plan (Section 11).

8.1 Cutover Runbook

Step	Action	Owner	Time	Status
C-1	Send maintenance window START notification to all teams	PM		
C-2	Confirm no long-running transactions on source (> 30 min)	DBA		
C-3	Stop all application write traffic to source Oracle DB	App		
C-4	Record final SCN: SELECT current_scn FROM v$database	DBA		
C-5	Confirm DMS CDC latency is < 5 seconds	DBA		
C-6	Wait for DMS to apply all pending transactions (latency = 0)	DBA		
C-7	Run final row count validation (Section 7.1 QUERY 1+2)	DBA		
C-8	Run final DMS validation check (Section 7.2)	DBA		
C-9	Stop DMS replication task	Cloud		
C-10	Re-enable all FK constraints on target RDS	DBA		
C-11	Rebuild any UNUSABLE indexes on target RDS	DBA		
C-12	Validate sequences are above max source PK values	DBA		
C-13	Compile all invalid objects on RDS	DBA		
C-14	Run application smoke test against RDS endpoint	App		
C-15	Update DNS / load balancer to point to RDS endpoint	Cloud		
C-16	Update connection strings in app config / Secrets Manager	Cloud		
C-17	Restart application servers with new RDS connection	App		

Step	Action	Owner	Time	Status
C-18	Verify application health checks pass on RDS	App		
C-19	Monitor application logs for DB errors (15 minutes)	App		
C-20	Enable RDS Multi-AZ (async, does not cause downtime)	Cloud		
C-21	Send cutover SUCCESS notification to all teams	PM		

8.2 Go / No-Go Decision Gate

At step C-14 (application smoke test), the following parties must confirm GO before proceeding to C-15 (DNS cutover):

Role	Decision	Signature	Time
DBA Lead	GO / NO-GO		
Application Owner	GO / NO-GO		
Cloud Eng Lead	GO / NO-GO		
Incident Manager	GO / NO-GO		

8.3 Cutover SQL Queries

```
-- QUERY 24: Confirm no active transactions before stopping app
SELECT s.sid, s.serial#, s.username, s.status,
       t.used_ublk, t.used_rec,
       ROUND((SYSDATE - t.start_date)*1440,1) AS duration_min
FROM   v$session s JOIN v$transaction t ON s.taddr = t.addr
ORDER BY duration_min DESC;
-- Must be 0 rows (or only DMS user sessions)

-- QUERY 25: Final SCN capture (record this value)
SELECT current_scn, TO_CHAR(SYSDATE,'DD-MON-YYYY HH24:MI:SS') AS cutover_time
FROM   v$database;

-- QUERY 26: Re-enable all FK constraints on target RDS
BEGIN
  FOR c IN (SELECT owner, table_name, constraint_name
            FROM   dba_constraints
            WHERE  constraint_type = 'R'
            AND    status = 'DISABLED'
            AND    owner = 'YOUR_SCHEMA')
  LOOP
    EXECUTE IMMEDIATE 'ALTER TABLE ' || c.owner || '.' ||
                      c.table_name || ' ENABLE CONSTRAINT ' || c.constraint_name;
```

```
  END LOOP;
END;
/

-- QUERY 27: Re-enable all triggers on target RDS
BEGIN
  FOR t IN (SELECT owner, trigger_name, table_name
            FROM   dba_triggers
            WHERE  status = 'DISABLED'
            AND    owner = 'YOUR_SCHEMA')
  LOOP
    EXECUTE IMMEDIATE 'ALTER TRIGGER ' || t.owner || '.' || t.trigger_name || '
ENABLE';
  END LOOP;
END;
/
```

9. Phase 5 — Post-Cutover Validation

9.1 Immediate Post-Cutover Checks (T+0 to T+1 hour)

```
-- QUERY 28: Final row count parity check (source vs RDS)
-- Run on source Oracle:
SELECT 'SOURCE' AS db, table_name, num_rows
FROM   user_tables ORDER BY table_name;

-- Run on target RDS:
SELECT 'TARGET_RDS' AS db, table_name, num_rows
FROM   user_tables ORDER BY table_name;

-- QUERY 29: Verify application can INSERT successfully
INSERT INTO schema.app_health_check (check_id, check_time, check_type)
VALUES (app_seq.NEXTVAL, SYSTIMESTAMP, 'POST_CUTOVER');
COMMIT;
SELECT * FROM schema.app_health_check WHERE check_type = 'POST_CUTOVER';

-- QUERY 30: Active sessions on RDS (confirm app is connected)
SELECT username, COUNT(*) AS session_count, status
FROM   v$session
WHERE  username NOT IN ('SYS','SYSTEM','RDSADMIN')
GROUP BY username, status
ORDER BY session_count DESC;

-- QUERY 31: Check for ORA errors in RDS alert log
SELECT message_text, originating_timestamp
FROM   v$diag_alert_ext
WHERE  originating_timestamp > SYSDATE - 1/24   -- last 1 hour
AND    message_text LIKE 'ORA-%'
ORDER BY originating_timestamp DESC;

-- QUERY 32: RDS storage utilization
SELECT tablespace_name,
       ROUND(used_space * 8192 / 1073741824, 2) AS used_gb,
       ROUND(tablespace_size * 8192 / 1073741824, 2) AS total_gb,
       ROUND(used_percent, 1) AS used_pct
FROM   dba_tablespace_usage_metrics
ORDER BY used_pct DESC;
```

9.2 Performance Validation

```
-- QUERY 33: Top 10 CPU-consuming SQL on RDS (post-cutover)
SELECT sql_id, elapsed_time/1000000 AS elapsed_sec,
       cpu_time/1000000 AS cpu_sec,
       executions, buffer_gets,
       SUBSTR(sql_text,1,80) AS sql_snippet
FROM   v$sql
WHERE  executions > 0
ORDER BY cpu_time DESC
FETCH FIRST 10 ROWS ONLY;
```

```
-- QUERY 34: Wait events on RDS
SELECT event, total_waits, time_waited_micro/1000000 AS wait_sec, wait_class
FROM   v$system_event
WHERE  wait_class != 'Idle'
ORDER BY time_waited_micro DESC
FETCH FIRST 20 ROWS ONLY;

-- QUERY 35: Blocking sessions
SELECT blocking_session, sid, serial#, wait_class,
       seconds_in_wait, state
FROM   v$session
WHERE  blocking_session IS NOT NULL
ORDER BY seconds_in_wait DESC;

-- QUERY 36: Undo/Temp usage
SELECT tablespace_name, status,
       ROUND(SUM(bytes)/1073741824,2) AS gb_used
FROM   dba_undo_extents
GROUP BY tablespace_name, status
ORDER BY tablespace_name;
```

9.3 Post-Cutover Checklist

Check Item	Criteria	Verified By	Result
Application writing to RDS successfully	INSERTs working		
Application reading from RDS successfully	SELECTs working		
Row counts match source snapshot	0 diff		
No ORA- errors in RDS alert log	0 ORA- errors		
Application error rate < pre-migration baseline	Within SLA		
Response time within 10% of pre-migration	p95 latency OK		
No blocking sessions	0 blocks		
Tablespace usage < 80%	< 80% full		
RDS automated backup completed	Backup confirmed		
RDS Multi-AZ standby in sync	Available		
Monitoring dashboards updated to RDS metrics	Updated		
Source Oracle DB in READ-ONLY or shutdown	Confirmed		

Check Item	Criteria	Verified By	Result
DMS task stopped cleanly	Stopped		

10. Phase 6 — Hypercare & Monitoring

10.1 Hypercare Period (T+0 to T+14 days)

Period	Monitoring Frequency	Focus Areas
T+0 to T+4 hours	Continuous	App errors, DB performance, connection counts, replication confirmation
T+4h to T+24h	Every 30 min	Tablespace growth, slow queries, ORA errors, backup verification
Day 2 to Day 7	Every 2 hours	Batch jobs, end-of-day processes, index usage, undo growth
Day 8 to Day 14	Every 4 hours	Monthly reports, scheduler jobs, AWR comparison vs. baseline

10.2 CloudWatch Alarms to Monitor

Alarm	Metric	Threshold	Action
High CPU	CPUUtilization	> 85% for 10 min	Investigate top SQL, consider scale up
Low memory	FreeableMemory	< 1 GB	Check memory hog processes, scale up
Storage critical	FreeStorageSpace	< 20 GB	Add storage or clean up tablespaces
Read latency high	ReadLatency	> 20ms avg	Check I/O patterns, instance type
Connection spike	DatabaseConnections	> 400	Check connection leak in app
Deadlocks	Deadlocks	> 0	Review locking SQL in v$session
Replication lag	AuroraBinlogReplicaLag	> 30 sec	Check Multi-AZ sync status

10.3 Daily DBA Validation Queries (Hypercare Period)

```
-- QUERY 37: Daily health check — run every morning
SELECT 'Sessions'        AS metric, COUNT(*) AS value FROM v$session WHERE status =
'ACTIVE'
UNION ALL
SELECT 'Blocking',       COUNT(*) FROM v$session WHERE blocking_session IS NOT NULL
UNION ALL
SELECT 'Long Queries',   COUNT(*) FROM v$session WHERE last_call_et > 300 AND status =
'ACTIVE'
UNION ALL
```

```
SELECT 'Undo Segments',  COUNT(*) FROM v$rollstat WHERE status = 'ONLINE';

-- QUERY 38: Tablespace trending
SELECT tablespace_name,
       ROUND(used_percent, 1) AS pct_used,
       ROUND((tablespace_size - used_space) * 8192 / 1073741824, 2) AS free_gb
FROM   dba_tablespace_usage_metrics
ORDER BY pct_used DESC;

-- QUERY 39: Failed scheduler jobs
SELECT job_name, status, error#, actual_start_date, run_duration
FROM   dba_scheduler_job_run_details
WHERE  status = 'FAILED'
AND    actual_start_date > SYSDATE - 1
ORDER BY actual_start_date DESC;

-- QUERY 40: Invalid objects (should remain 0)
SELECT COUNT(*) AS invalid_obj_count
FROM   user_objects WHERE status = 'INVALID';
```

11. Rollback Plan

ROLLBACK TRIGGER: Invoke rollback if ANY of these conditions occur during or after cutover: (1) Critical application errors > 5% error rate, (2) Data integrity issues found on RDS, (3) Performance degradation > 50% vs baseline, (4) RDS instance unavailable, (5) DBA Lead or Application Owner declares NO-GO.

11.1 Rollback Decision Authority

The DBA Lead or Application Owner may independently call a rollback. No consensus required. Time to rollback decision must be < 30 minutes post-cutover.

11.2 Rollback Runbook

Step	Action	Owner	ETA
R-1	Declare rollback — notify all teams immediately	DBA Lead	T+1 min
R-2	Revert DNS / load balancer to point back to source Oracle	Cloud	T+5 min
R-3	Restart application with original Oracle connection string	App	T+10 min
R-4	Verify application connects to source Oracle successfully	App	T+12 min
R-5	Confirm source Oracle DB is in READ-WRITE mode	DBA	T+13 min
R-6	Identify and replay any transactions written to RDS post-cutover	DBA	T+30 min
R-7	Run row count comparison source vs RDS to find delta	DBA	T+35 min
R-8	Apply delta transactions to source if needed	DBA	T+60 min
R-9	Verify application functioning normally on source Oracle	App	T+65 min
R-10	Send rollback COMPLETE notification. Schedule post-mortem.	PM	T+70 min

11.3 Delta Recovery Query

```
-- QUERY 41: Find transactions written to RDS after cutover SCN
-- (Run on RDS target to identify data to replay on source)
SELECT table_name, operation, row_data, commit_timestamp
FROM   schema.dms_cdc_audit_log    -- if audit log table exists
```

```
WHERE  commit_timestamp > TO_TIMESTAMP('&cutover_time','YYYY-MM-DD HH24:MI:SS')
ORDER BY commit_timestamp;

-- QUERY 42: Identify rows written to RDS post-cutover
-- (Use row-level audit or last_updated timestamp column)
SELECT * FROM schema.orders
WHERE  last_updated_at > TO_TIMESTAMP('&cutover_time','YYYY-MM-DD HH24:MI:SS')
ORDER BY last_updated_at;
```

12. SQL Reference — Complete Validation Query Library

This section consolidates ALL validation queries in one place for quick reference during migration execution.

12.1 Source Oracle Health Queries

```
-- SOURCE-01: Database version and mode
SELECT * FROM v$version;
SELECT name, db_unique_name, log_mode, open_mode, database_role FROM v$database;

-- SOURCE-02: Archive log status
SELECT sequence#, name, first_time, next_time, archived, status
FROM   v$archived_log
ORDER BY sequence# DESC FETCH FIRST 20 ROWS ONLY;

-- SOURCE-03: Redo log configuration
SELECT group#, members, bytes/1048576 AS size_mb, status FROM v$log;

-- SOURCE-04: Active sessions
SELECT username, status, COUNT(*) AS cnt
FROM   v$session WHERE type = 'USER'
GROUP BY username, status ORDER BY cnt DESC;

-- SOURCE-05: Long-running transactions
SELECT s.username, s.sid, s.serial#,
       ROUND((SYSDATE - t.start_date)*1440,1) AS duration_min,
       t.log_io, t.used_ublk
FROM   v$session s JOIN v$transaction t ON s.taddr = t.addr
ORDER BY duration_min DESC;

-- SOURCE-06: Database size
SELECT ROUND(SUM(bytes)/1073741824,2) AS total_db_size_gb
FROM   dba_data_files;

-- SOURCE-07: Tablespace usage on source
SELECT df.tablespace_name,
       ROUND(df.bytes/1073741824,2)                AS allocated_gb,
       ROUND((df.bytes - fs.free_bytes)/1073741824,2) AS used_gb,
       ROUND(fs.free_bytes/1073741824,2)           AS free_gb
FROM  (SELECT tablespace_name, SUM(bytes) AS bytes FROM dba_data_files GROUP BY
tablespace_name) df
JOIN  (SELECT tablespace_name, SUM(bytes) AS free_bytes FROM dba_free_space GROUP BY
tablespace_name) fs
      ON df.tablespace_name = fs.tablespace_name
ORDER BY used_gb DESC;
```

12.2 DMS Validation Queries

```
-- DMS-01: List all DMS tasks
```

```
aws dms describe-replication-tasks \
  --query 'ReplicationTasks[*].{Name:ReplicationTaskIdentifier,Status:Status,Type:MigrationType}'

-- DMS-02: Table statistics
aws dms describe-table-statistics \
  --replication-task-arn <ARN> \
  --query
'TableStatistics[*].{Schema:SchemaName,Table:TableName,FullLoadRows:FullLoadRows,Errors:FullLoadErrorRow

-- DMS-03: Replication instance status
aws dms describe-replication-instances \
  --query
'ReplicationInstances[*].{ID:ReplicationInstanceIdentifier,Status:ReplicationInstanceStatus,CPU:AutoMinc

-- DMS-04: CloudWatch CDC lag
aws cloudwatch get-metric-statistics --namespace AWS/DMS \
  --metric-name CDCLatencyTarget \
  --dimensions Name=ReplicationInstanceIdentifier,Value=<INSTANCE_ID> \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 --statistics Average Maximum

-- DMS-05: View DMS task logs
aws logs filter-log-events \
  --log-group-name /aws/dms/tasks/<TASK_ID> \
  --filter-pattern 'ERROR' \
  --start-time $(date -d '2 hours ago' +%s000)
```

12.3 Target RDS Validation Queries

```
-- RDS-01: Verify database mode
SELECT name, open_mode, db_unique_name FROM v$database;

-- RDS-02: Object and status summary
SELECT object_type,
       COUNT(*) total,
       SUM(CASE WHEN status='VALID' THEN 1 ELSE 0 END) valid,
       SUM(CASE WHEN status='INVALID' THEN 1 ELSE 0 END) invalid
FROM   user_objects GROUP BY object_type ORDER BY object_type;

-- RDS-03: All constraints summary
SELECT constraint_type, status, validated, COUNT(*)
FROM   user_constraints GROUP BY constraint_type, status, validated;

-- RDS-04: Sequence check
SELECT sequence_name, last_number, increment_by, cache_size
FROM   user_sequences ORDER BY sequence_name;

-- RDS-05: Index check
SELECT index_name, table_name, index_type, status, uniqueness
FROM   user_indexes ORDER BY table_name;

-- RDS-06: User-defined types
```

```
SELECT type_name, typecode, attributes, methods
FROM   user_types ORDER BY type_name;

-- RDS-07: Synonyms
SELECT synonym_name, table_owner, table_name
FROM   user_synonyms ORDER BY synonym_name;

-- RDS-08: Database links
SELECT db_link, username, host FROM user_db_links;

-- RDS-09: Grants check
SELECT privilege, table_name, grantee, grantable
FROM   user_tab_privs_made ORDER BY table_name;

-- RDS-10: NLS settings (must match source)
SELECT parameter, value FROM nls_database_parameters
WHERE  parameter IN ('NLS_CHARACTERSET','NLS_NCHAR_CHARACTERSET',
                     'NLS_LANGUAGE','NLS_TERRITORY','NLS_DATE_FORMAT');

-- RDS-11: RDS-specific parameter check
SELECT name, value FROM v$parameter
WHERE  name IN ('processes','sessions','open_cursors',
                'undo_retention','log_buffer','enable_goldengate_replication')
ORDER BY name;

-- RDS-12: Alert log errors (last 24 hours)
SELECT originating_timestamp, message_text
FROM   v$diag_alert_ext
WHERE  originating_timestamp > SYSDATE - 1
AND    message_text LIKE 'ORA-%'
ORDER BY originating_timestamp DESC FETCH FIRST 50 ROWS ONLY;
```

13. Troubleshooting Guide

Error / Symptom	Root Cause	Resolution
ORA-01031: insufficient privileges	DMS user missing grants	Re-run Section 5.1 privilege grants; check V$ views explicitly granted
Supplemental logging not enabled	CDC cannot read change vectors	Run ALTER DATABASE ADD SUPPLEMENTAL LOG DATA as SYSDBA
LOB data truncated / missing	DMS using Limited LOB mode (32KB cap)	Switch to Full LOB mode in task settings; expect slower throughput
ORA-01555: snapshot too old during full load	UNDO_RETENTION too low; large table scan overlaps undo expiry	Increase undo_retention to 86400 on source; schedule load off-peak
Primary key violation on INSERT after cutover	Sequence not advanced on target	Run Section 7.5 QUERY 19 to advance sequence above max PK
CDC latency continuously increasing	DMS instance undersized or network bottleneck	Scale up DMS replication instance; check Direct Connect utilization
Table validation stuck in Pending	Source table has high DML activity during validation	Run validation during low-traffic window or freeze writes
ORA-12899: value too large for column	Character set mismatch (e.g. US7ASCII source, AL32UTF8 target)	Verify NLS_CHARACTERSET matches; use SCT to adjust column sizes
Missing rows on target after full load	Table excluded by mapping rules, or DMS error rows	Check task error log; review mapping rules; check FullLoadErrorRows
Application ORA-00018: max sessions exceeded	PROCESSES/SESSIONS parameter too low on RDS	Increase via RDS parameter group; reboot required for static params
Indexes UNUSABLE on target after load	DMS deferred index maintenance during bulk load	Run ALTER INDEX ... REBUILD ONLINE for each unusable index
Stored procedures fail on RDS	External dependencies (directory objects, DB links) not migrated	Manually create directory objects; update DB link definitions for RDS

14. Communication & Escalation Plan

14.1 Communication Timeline

When	Who Notifies	Audience	Channel	Message
T-5 business days	PM	All stakeholders	Email	Maintenance window reminder with runbook link
T-24 hours	PM	App teams, ops	Email + Slack	Final reminder; cutover checklist attached
T-1 hour	DBA Lead	Bridge call open	Phone	Standby call bridge open; team check-in
Cutover start	PM	All	Slack	Maintenance window START — DB in maintenance
GO decision	DBA Lead	App + Ops	Bridge	GO confirmed — switching traffic to RDS
Cutover complete	PM	All	Email	Migration COMPLETE; RDS is now production
Rollback invoked	DBA Lead	All + Management	All	ROLLBACK initiated — ETA to restore [X] min
T+24h	DBA Lead	Management	Email	24-hour post-migration health report

14.2 Escalation Matrix

Severity	Scenario	Escalate To	Response SLA
P1 — Critical	Migration blocked, production down, data loss	CTO, VP Eng, AWS Support Enterprise	< 15 min
P2 — High	Data mismatch, app error rate > 5%, rollback triggered	DBA Director, App Director	< 30 min
P3 — Medium	Performance degradation, single table issue	DBA Lead, Cloud Lead	< 2 hours
P4 — Low	Minor issue, no user impact	DBA Team	Next business day

15. Sign-off & Approval

15.1 Migration Execution Sign-off

Milestone	Signed Off By	Role	Date & Time	Outcome
Phase 3 — Pre-Cutover Validation Complete		DBA Lead		GO / NO-GO
Phase 3 — Application Smoke Test on RDS		App Owner		GO / NO-GO
Phase 4 — Go/No-Go Decision (C-14)		All Leads		GO / NO-GO
Phase 4 — Cutover to RDS Complete (C-21)		DBA Lead		
Phase 5 — Post-Cutover Validation Complete		DBA Lead		PASS / FAIL
Phase 5 — Application Sign-off on RDS		App Owner		PASS / FAIL
Phase 6 — Hypercare Period Start		DBA Lead		
Phase 6 — Hypercare Period End / Handover		DBA + Ops Lead		
Final Migration Closure		CAB		CLOSED

15.2 Post-Migration Lessons Learned

Category	What Went Well	What Could Improve	Action Owner	Due Date
Planning				
Execution				
Validation				
Communication				
Rollback				
Tooling				

END OF DOCUMENT

Production DB Migration SOP v2.0 | AWS DMS + Oracle RDS | May 2026